

Arquitetura Consolidada

Pós B-38 · B-39 · B-48 · B-49

Inventário arquitetural, padrões identificados, duplicações eliminadas, componentes consolidados, 7 ADRs, diretrizes oficiais e preparação de documentação para o B-46. 7 ADRs formalizando decisões já implementadas. Nenhum componente criado sem benefício comprovado.

RESULTADO DESTA REVISÃO

Data: 2026-08-01
Projeto: AtendeHub (apps/api — NestJS · Prisma · PostgreSQL · Bull · Redis)
Status: ' Concluída

1. Resumo executivo

Após o fechamento de B-38 (storage privado), B-39 (retry/DLQ de webhook), B-48 (idempotência de Message) e B-49 (idempotência de Conversation), esta Architecture Consolidation Review (ACR) mapeou o backend inteiro do AtendeHub para consolidar padrões recorrentes, eliminar duplicação real e preparar tecnicamente o backlog B-40 a B-47 — sem desenvolver nenhuma funcionalidade nova.

Inventário completo: 20 módulos, 30 services, 18 controllers, 4 filas Bull, 1 interceptor global, 3 guards, 4 decorators, 1 middleware, 1 gateway Socket.IO, e — achado central desta revisão — 5 clients Redis instanciados separadamente no projeto, sem nenhum módulo compartilhado.

Duas duplicações reais foram encontradas e consolidadas nesta sessão: o predicado de violação de constraint única (`!P2002``), repetido byte-a-byte em `MessageService` e `ConversationService`; e o formato de log estruturado, que já existia mas só era usado dentro do módulo de webhook, enquanto as duas correções de idempotência (B-48/B-49) logavam colisão com string ad-hoc. Um terceiro achado, maior — 5 clients Redis, 2 deles com configuração idêntica e sem diferenciação técnica — foi analisado, documentado e deliberadamente NÃO consolidado nesta sessão (código de segurança em produção, risco desproporcional como efeito colateral de uma revisão), com recomendação explícita de execução dentro do escopo do item B-46, que já toca essa mesma área.

7 ADRs novas formalizam decisões já implementadas (idempotência, persistência atômica, concorrência, retry, DLQ, tratamento de erros, observabilidade). Da lista de componentes sugerida para avaliação, nenhum foi criado sem necessidade comprovada — cada não-criação tem justificativa técnica registrada. Suíte completa (367/367), E2E (7/7) e build/lint/typecheck seguem limpos após toda a consolidação.

2. Objetivo

Consolidar os padrões arquiteturais do projeto, eliminar duplicações reais (com evidência de código, não suposição), padronizar diretrizes de desenvolvimento, formalizar decisões já tomadas como ADR, e preparar tecnicamente o restante do backlog (B-40 a B-47) — identificando dependências e reaproveitamento de padrão, sem alterar regra de negócio nem introduzir abstração sem benefício comprovado.

3. Inventário arquitetural

Categoria	Quantidade	Observação
Módulos	22 (20 de domínio + Prisma/Storage globais)	
Services	30	Nenhuma camada de repository — Prisma injetado direto
Controllers	18	
Filas Bull	4	webhook, webhook-dlq, sla-check, auto-attendance-inactivity
Interceptors	1	MediaPresignInterceptor (global, B-38)
Guards	3	Todos de auth, registrados globalmente
Decorators customizados	4	Todos em modules/auth/decorators/
Middlewares	1	RequestIdMiddleware
Gateways WebSocket	1	EventsGateway, namespace /ws
Cron jobs	1	RefreshTokenCleanupService, diário
Clients Redis independentes	5	TokenBlacklist, AutoAttendanceSession, Health, RedisIoAdapter, BullMQ — achado central desta ACR
Integrações externas	4	Evolution API (axios), MinIO (SDK), Sentry, PostgreSQL/Prisma

Inventário completo (arquivo por arquivo) preservado no histórico desta sessão; resumo consolidado em docs/04-Arquitetura/padroes-consolidados-2026-08.md.

4. Padrões identificados

Padrão	Onde	Situação
Idempotência	MessageService (B-48), ConversationService (B-49)	Consistente — mesma receita, recuperação por domínio
Persistência atômica	UNIQUE de coluna (Message) e índice parcial (Conversation)	Consistente com a invariante de cada recurso
Retry	webhook (política própria) vs. default global (SLA, auto-atendimento)	Divergência intencional, documentada (ADR-0005)
DLQ	Só fila webhook	Escopo intencional (ADR-0006)
Classificação de erro	ValidationPipe / SentryExceptionFilter / classifyWebhookError	3 camadas complementares (ADR-0007)
Log estruturado	shared/logging/structured-log.util.ts	Consolidado nesta ACR — antes só em webhook
Correlação	AsyncLocalStorage (HTTP) + threading explícito (fila)	Formalizado nesta ACR (ADR-0008)
Estado efêmero com TTL (Redis)	TokenBlacklistService, AutoAttendanceSessionService	Duplicação real — ver seção 5
Repository	Não existe	Decisão implícita consolidada — Prisma É o ponto de acesso a dado

5. Duplicações encontradas

#	Local	Qtd	Risco	Ação
1	Predicado P2002 (instanceof + código)	2	Baixo	Consolidado — isUniqueConstraintViolation
2	Log de colisão com string ad-hoc	2	Médio (parsing de log)	Consolidado — formatStructuredLog reusado
3	new Redis({host,port,password}) idêntico	2 (tende a 3+ via B-46)	Médio (conexões redundantes)	Documentado, endereçado ao B-46 — não executado agora
4	Extração de telefone de JID	0 (já resolvido em B-39)	—	Nenhuma — referência de consolidação bem-sucedida

6. Comparação das alternativas arquiteturais — casos centrais

6.1 Idempotência (ADR-0002)

7 alternativas comparadas (mutex, Redis lock, upsert nativo, UNIQUE+P2002, SELECT FOR UPDATE, SERIALIZABLE, não fazer nada). Escolhida UNIQUE/índice parcial + P2002: única alternativa que garante atomicidade sob qualquer grau de concorrência sem infraestrutura nova e sem depender de nenhum código futuro lembrar de usar um lock.

6.2 Client Redis compartilhado

(achado desta ACR, não uma ADR própria — decisão de escopo, não de arquitetura)

Analizado se os 5 clients deveriam consolidar em 1. Conclusão: NÃO — 3 dos 5 têm requisito técnico real de ficarem separados (Health precisa falhar rápido; RedisIoAdapter é exigência da lib; BullMQ gerencia a própria conexão). Só 2 (TokenBlacklistService, AutoAttendanceSessionService) são duplicação real sem diferenciação — candidatos a um módulo compartilhado, ação recomendada para dentro do escopo do B-46.

6.3 AtomicPersistenceHelper genérico (ADR-0003)

Comparado extrair um helper cross-model vs. manter cada domínio com seu próprio try/catch. Decisão: manter separado — a consulta de recuperação já diverge estruturalmente entre Message (lookup único) e Conversation (índice parcial via findFirst), então um helper genérico só economizaria a checagem de erro (já extraída) à custa de generics difíceis de ler.

7. Componentes consolidados nesta sessão

Componente	Arquivo	Consumidores
isUniqueConstraintViolation	shared/prisma/prisma-errors.util.ts	MessageService, ConversationService
formatStructuredLog / WORKER_ID	shared/logging/structured-log.util.ts (relocado)	WebhookProcessor, WebhookDlqService, MessageService, ConversationService

Componentes avaliados e conscientemente NÃO criados

- AtomicPersistenceHelper / IdempotencyService — ver ADR-0003. 2 consumidores, recuperação já divergente.
- RetryPolicy / RetryClassifier genérico — cada fila tem requisito real diferente (ADR-0005/0007).
- ConcurrencyGuard / DomainInvariantValidator — a garantia já vem do banco; validador de aplicação seria 2ª fonte de verdade competindo com a constraint.
- WebhookPipelineHelper — nenhuma lógica repetida entre os estágios do pipeline que justifique abstração própria.
- RepositoryBase — o projeto não usa repository; nenhuma duplicação de query encontrada entre services que justifique introduzir a camada agora.
- TraceContext — já existe de fato (AsyncLocalStorage + threading explícito); só precisava de documentação (ADR-0008), não de componente novo.

- MetricsService genérico — WebhookMetricsService é hoje o único consumidor real; generalizar antes de um 2º caso seria especulativo.

8. Diretrizes oficiais de desenvolvimento

Publicadas em docs/06-Backend/padroes-idempotencia-concorrencia-retry.md, docs/25-Observabilidade/logs-estruturados-e-correlacao.md e docs/23-Testes/Stress/concorrencia-real.md.
Resumo:

- Idempotência: nunca findFirst()+create() sozinho; modelar como UNIQUE (coluna inteira) ou índice parcial (unicidade condicional); create() + catch(P2002) via isUniqueConstraintViolation; isNew explícito quando o consumidor precisa distinguir novo de retry.
- Retry: transitório !' deixa o Bull re-tentar; permanente !' job.discard() imediato; não reconhecido !' trata como transitório (nunca desiste sozinho por classificação errada).
- Backoff exponencial: só quando a causa provável é instabilidade temporária de serviço externo — não usar backoff longo pra falha que é quase sempre bug de código interno.
- DLQ: só quando a fonte é externa e historicamente instável, perder o evento tem custo alto, e reprocessar depois faz sentido.
- Transação Prisma: só para múltiplas escritas que precisam ser tudo-ou-nada juntas — nunca

como substituto de
constraint única.

- Log estruturado:
sempre formatStructuredLog(evento, campos),
nunca string interpolada à
mão.

- Correlação:
AsyncLocalStorage
funciona sozinho dentro
do ciclo HTTP;
atravessando uma
fronteira de fila, threadar
o id explicitamente no
payload do job.

- Teste de concorrência
real: Promise.all (nunca
sequencial) contra
Postgres real, progressão
2!100, medir contagem de
linhas + isNew + IDs
distintos convergindo +
zero exceção.

9. ADRs criados

#	Título	Decisão em 1 frase
0002	Estratégia de idempotência	UNIQUE/índice parcial + P2002, nunca lock de aplicação
0003	Persistência atômica	Sem helper genérico — 2 consumidores, recuperação já divergente
0004	Concorrência e seus testes	Unitário (mock) + script real (Promise.all) — nenhum substitui o outro
0005	Estratégia de retry	Política por fila — default global + override só onde justificado
0006	Dead Letter Queue	Só onde a fonte é externa e instável — não em toda fila por padronização
0007	Tratamento de erros	3 classificadores complementares, não 1 genérico
0008	Observabilidade	AsyncLocalStorage no HTTP; threading explícito atravessando fila

Índice completo em docs/05-ADR/INDICE.md — inclui também a lista de ADRs previstas (decisões futuras ainda não formalizadas), renumerada nesta sessão para não colidir com 0002-0008.

10. Estrutura documental

O projeto já tem uma estrutura de documentação numerada (41 áreas, decisão registrada em ADR-0001) — as pastas genéricas sugeridas (docs/adr/, docs/patterns/, docs/guidelines/, docs/concurrency/, docs/idempotency/, docs/retries/, docs/observability/, docs/testing/) NÃO foram criadas como pastas paralelas: mapeadas nas áreas numeradas já existentes, evitando duplicar a própria governança documental que esta ACR deveria reforçar.

Pedido	Mapeado para	Situação
docs/architecture/	docs/04-Arquitetura/	Já existia — novo arquivo padroes-consolidados-2026-08.md adicionado
docs/adr/	docs/05-ADR/	Já existia — 7 ADRs novas + índice atualizado
docs/patterns/, docs/guidelines/	docs/06-Backend/	Novo arquivo padroes-idempotencia-concorrencia-retry.md

docs/concurrency/, docs/idempotency/, docs/retries/	docs/06-Backend/ + docs/23-Testes/Stress/	Cobertos pelo guia de backend + guia de teste de concorrência
docs/observability/	docs/25-Observabilidade/	Novo arquivo logs-estruturados-e-correlacao.md
docs/testing/	docs/23-Testes/	Já existia — novo arquivo concorrencia-real.md em Stress/

11. Impacto sobre o backlog B-40 a B-47

Nenhuma prioridade original reclassificada — nenhum achado desta ACR eleva ou rebaixa a severidade atribuída na auditoria de 2026-07-28.

Item	Reaproveita padrão desta ACR?	Dependência/conflito
B-40 (escopo de permissão)	Não	Domínio de autorização, independente
B-41 (RLS multi-tenant)	SIM — AsyncLocalStorage (ADR-0008)	A mesma fronteira HTTP! "fila documentada no ADR-0008 vale para companyId — threading explícito também seria necessário dentro de um processor
B-42 (preview de mídia sem legenda)	Não	Bug pontual de webhook.service.ts, independente
B-43 (headers nginx)	Não	Infra, fora do escopo desta ACR (backend)
B-44 (Docker root/healthcheck)	Não	Infra, fora do escopo desta ACR
B-45 (Swagger/OpenAPI)	Não diretamente	ValidationPipe (ADR-0007) já é a base de validação que os DTOs anotados exporiam
B-46 (estado em memória)	SIM — client Redis compartilhado	Implementar seguindo literalmente "mesmo padrão" criaria um 3º client duplicado — recomendado extrair módulo compartilhado neste item, migrando os 2 existentes junto
B-47 (lint frontend)	Não	Frontend, fora do escopo desta ACR (backend)

12. Plano de evolução arquitetural

- Curto prazo (junto do B-46): extrair RedisModule/client de app-data compartilhado, migrar TokenBlacklistService e AutoAttendanceSessionService, cobrir os 3 com regressão.
- Médio prazo (se volume de colisões de P2002 em produção justificar): threading de requestId até MessageService/ConversationService (residual documentado no ADR-0008).
- Médio prazo (se múltiplas réplicas reais entrarem em produção): agregação central de WebhookMetricsService (Redis), mesma classe de problema do B-46.
- Longo prazo: formalizar as ADRs previstas já

listadas em docs/05-ADR/INDICE.md (Modular Monolith, RLS, Bull!BullMQ, etc.) conforme o roadmap de evolução Enterprise avançar.

13. Não conformidades

#	Não conformidade	Severidade	Status
1	Predicado P2002 duplicado em 2 services	Baixa	CORRIGIDA
2	Log de colisão com formato inconsistente (string ad-hoc vs. estruturado)	Média	CORRIGIDA
3	5 clients Redis independentes, 2 idênticos sem diferenciação técnica	Média	DOCUMENTADA — endereçada ao B-46, não corrigida nesta sessão (código de segurança em produção, risco desproporcional fora de contexto)
4	requestId não threadado até os logs de colisão de idempotência (Message/Conversation)	Baixa	REGISTRADA — ADR-0008, sem ação até haver volume real que justifique
5	WebhookMetricsService não agrega entre réplicas	Baixa	REGISTRADA — mesma classe do B-46, sem ação nesta sessão

14. Recomendações futuras

- Executar a consolidação de Redis (achado #3) como parte do B-46, não isoladamente.
- Reavaliar threading de requestId (achado #4) quando houver dado real de volume de colisão em produção.
- Ao formalizar qualquer ADR da lista "previstas", seguir o template já existente (ADR-0000) e numerar sequencialmente a partir do maior número em uso.
- Repetir esta ACR (ou uma versão focal) após o fechamento de B-40/B-41/B-46 — são os 3 itens do backlog atual com maior potencial de introduzir padrão novo (guard de escopo, RLS, módulo Redis).

15. Checklist de conformidade

- Inventário arquitetural completo e factual
- Padrões mapeados com evidência de código (não suposição)
- Duplicações reais identificadas com local/quantidade/risco

- Toda consolidação comparada com alternativas antes de decidir
- Nenhum componente criado sem benefício comprovado
- Nenhuma abstração introduzida sem necessidade real
- 7 ADRs criadas seguindo o template já existente do projeto
- Diretrizes oficiais publicadas nas pastas numeradas corretas (não pastas paralelas)
- Backlog B-40 a B-47, analisado item a item, com justificativa
- **REVISÃO CONCLUÍDA**
 - Nenhuma regra de negócio alterada
 - A arquitetura do backend do AtendeHub, após B-38/B-39/B-48/B-49, está consolidada: os padrões reais de compatibilidade com B-38/B-39/B-46/B-49 preservada (suíte + E2E sem regressão)
 - Idempotência, retry, DLO, tratamento de erro e observabilidade estão mapeados, documentados como ADR e como diretriz oficial de desenvolvimento, com as duas duplicações genuínas eliminadas e o achado maior (clients Redis) endereçado ao item de backlog correto em vez de forçado nesta sessão. O backlog B-40 a B-47 foi baseado em uma única documentação arquitetural real com esta revisão, e ambos já carregam a orientação necessária. Nenhuma funcionalidade nova foi desenvolvida; nenhuma regra de negócio foi alterada; suíte e E2E permanecem 100% verdes.

16. Conclusão técnica

A base técnica está apta a orientar as próximas fases do projeto.

Documento gerado a partir de evidências coletadas ao vivo durante a sessão de revisão (2026-08-01) — inventário real do código, greps e leituras diretas, sem dado fabricado.